

Redes Neuronales Artificiales para la clasificación de imágenes.

Miguel García Bohigues

Enero 2023

1 Introducción

El objetivo de este trabajo es desarrollar dos modelos de clasificación basados en redes neuronales capaces de identificar elementos que aparecen en ciertas imágenes.

Concretamente, los conjuntos de datos con los que vamos a tratar van a ser MNIST y CIFAR.

- El primero de estos es un conjunto de imágenes de dígitos manuscritos, del 0 al 9.
- El segundo trata de una colección de imágenes de diferentes objetos, tales como perros, aviones, coches, barcos, etc.

La complejidad de ambos problemas es diferente, por lo tanto vamos a separarlos en dos secciones. Primero hablaremos sobre el proceso para entrenar el mejor clasificador para MNIST. Posteriormente comentaremos lo propio para CIFAR.

2 MNIST

MNIST es un dataset muy conocido entre el mundo docente y de investigación, aunque en este último ha ido perdiendo protagonismo. Consta de imágenes en escala de grises de números escritos a mano del 0 al 9. Nuestro objetivo con este dataset es conseguir un modelo cuya precisión sea del 99.30% o más.

Para ello, inicialmente partimos de una topología simple, para tener una base sobre la que poder trabajar. Ésta se componía de una capa de aplanado que convierte imágenes en una lista de 784 elementos. Tras esta, 2 capas ocultas, cada una con 1024 neuronas y todas ellas con *ReLU* como función de activación. Como algoritmo de optimización se escogió descenso por gradiente estocástico y finalmente, en la capa de salida presentamos 10 neuronas con *SoftMax* como función de activación. De esta manera generábamos una densidad de probabilidad que podíamos utilizar para clasificar.

Con esto conseguimos que el modelo aprendiese el conjunto de entrenamiento a la perfección (99.96%) sin embargo, el resultado sobre el conjunto de prueba estaba aún muy lejos de lo que queríamos conseguir, con sólo un 97.85% de acierto. Como vemos, el sistema tiene potencia para aprender el conjunto de entrenamiento pero no es capaz de generalizar lo aprendido.

Como intento de mejora, colocamos una normalización a nivel de batch, tanto antes como después de la función de activación. El mejor de ambos resultados ha sido la normalización antes de la función de activación, mejorando el resultado base en un 0.3% (98.13%)

Vemos que con ello hemos mejorado algo, pero seguimos lejos de lo que deseamos obtener. Como siguiente paso, agregamos ruido gaussiano antes de la función de activación. Como manera de “forzar” la generalización del modelo. Para paliar la posible pérdida que pueda tener el modelo, hemos cambiado la topología, pasando a tener 1 capa de entrada, 3 ocultas con 1024 neuronas y la capa de salida, además de incrementar las épocas de entrenamiento.

El modelo resultante mostró un empeoramiento en el aprendizaje del conjunto de entrenamiento, pero una mejoría en la precisión con respecto al conjunto de prueba. En la siguiente imagen se puede ver el resultado obtenido:

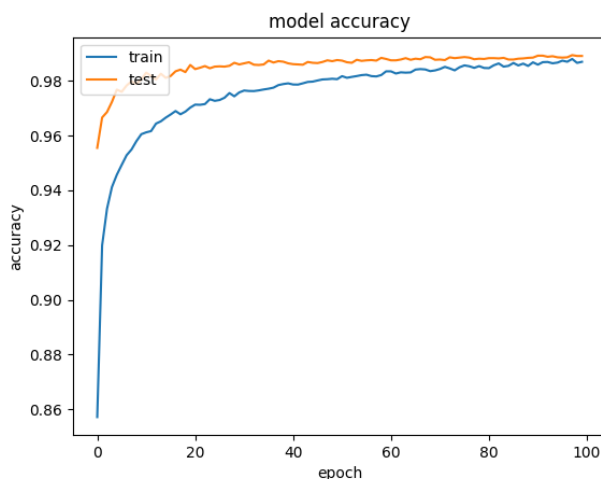


Figure 1: Proceso de entrenamiento con ruido Gaussiano

Como vemos, el sistema ha generalizado, pero quizás en exceso. Hemos conseguido reducir el gap de separación entre las curvas de entrenamiento y prueba, pero no hemos conseguido acercarnos al objetivo de precisión.

Para paliar esta pérdida, cambiamos la topología, brindando más potencia al modelo. Pasamos a 3 capas ocultas con 1024 neuronas cada una. Como optimización cambiamos SGD por ADAM y, agregamos un dropout incremental: 10% en la 2ª capa oculta, 15% en la 3ª y 20% en la 4ª. Con esto conseguimos

el mejor resultado hasta la fecha: 98.95%, ya mas cerca de romper la primera barrera, la del 99%.

Para hacer esto, decidimos agregar data augmentation al modelo. Generando muestras derivadas de las que tenemos moviéndolas ligeramente de izquierda a derecha y agregando un ruido gaussiano incremental también a medida que nos acercamos a la capa de salida. Después de varias pruebas para conocer los mejores parámetros de ruido, dropout y rango de data augmentation, el modelo obtiene un 99.07% de acierto en entrenamiento y un 99.33% en el conjunto de prueba, superando el objetivo que nos habíamos propuesto.

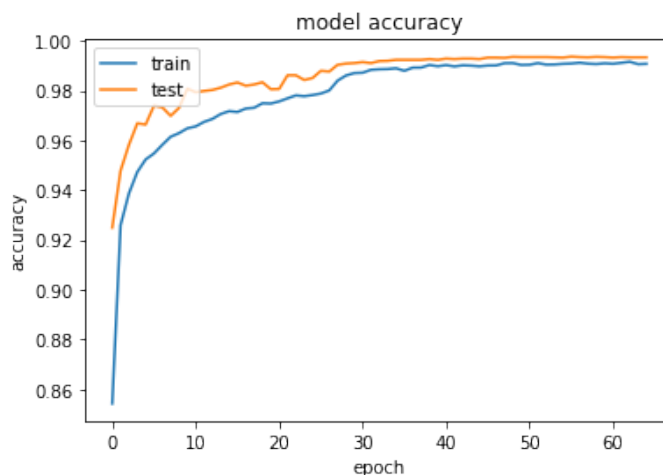


Figure 2: Proceso de entrenamiento del mejor modelo

3 CIFAR

CIFAR presenta el mismo problema que MNIST en el momento de clasificar: identificar cuál de los 10 objetos que tiene aparece en la imagen que está viendo. No obstante, en este caso el problema cambia, puesto que disponemos de 3 canales de color y de una imagen entera (puesto que en este escenario carece de sentido convertir la imagen en una lista de elementos). Por lo tanto, vamos a ir más allá a la hora de generar nuestra red neuronal base. En este caso vamos a utilizar capas convolucionales para poder identificar formas en las imagenes. Para ello hemos definido un bloque convolucional formado por un filtro 2D, normalización de batch, ruido Gaussiano, *ReLU* como función de activación y MaxPooling de 2x2. En la red más básica hemos partido con 5 de estas capas aplicando 32, 64, 128, 256 y 512 filtros respectivamente, tras estas capas hemos incluido una capa oculta de 512 neuronas también con *ReLU* y finalmente la capa de salida con 10 neuronas y *SoftMax*. Todos los experimentos se han hecho con reducción del factor de aprendizaje *on plateau*, con el optimizador Adam y

con un generador de datos que arbitrariamente movía las imágenes un 15% a derecha e izquierda, las rotaba hasta 15° o las volteaba horizontalmente.

La tasa de acierto que tenemos como referencia para este caso es del 92%. Con esta primera aproximación apenas conseguimos superar el 84.75% en test. Claro está que la red ha conseguido detectar ciertas formas en las imágenes de entrada, pero carece de potencia para poder aprender el dataset con el que estamos trabajando.

Como siguiente iteración, hemos modificado el bloque convolucional, parametrizando el uso del max pooling. De esta manera, podemos aplicar varios filtros convolucionales antes de reducir la dimensión de la imagen. Con esta idea, hemos aplicado una topología muy común en visión por computador, la Visual Geometry Group o más conocida como VGG. Estas topologías se basan precisamente en este principio de aplicar varias convoluciones antes de realizar el pooling. De esta manera, nuestra topología ha quedado:

$$\begin{aligned} & Conv(32) \rightarrow Conv(32) \rightarrow MaxP(2) \rightarrow Conv(64) \rightarrow Conv(64) \rightarrow MaxP(2) \rightarrow \\ & Conv(128) \rightarrow Conv(128) \rightarrow Conv(128) \rightarrow MaxP(2) \rightarrow Conv(256) \rightarrow Conv(256) \rightarrow \\ & Conv(256) \rightarrow MaxP(2) \rightarrow Conv(512) \rightarrow Conv(512) \rightarrow Conv(512) \rightarrow MaxP(2) \rightarrow \\ & Fcon(512) \rightarrow Fcon(10) \rightarrow softMax \end{aligned}$$

Este cambio en la topología ha ofrecido una mejora sustancial de los resultados, obteniendo un 92.42% de acierto sobre el conjunto de entrenamiento y un 89.58% sobre el conjunto de prueba. A pesar del incremento en el acierto y la mayor capacidad de la red neuronal para aprender el conjunto de datos, aún vemos como nos sigue faltando capacidad para aprender.

En un intento por paliar esta situación, agregamos al final del modelo dos capas ocultas más. Ambas fully connected con 512 neuronas cada una. Para evitar el sobre-ajuste y la pérdida de gradiente por el aumento de la profundidad de la topología, hemos agregado a la parte fully connected una capa de normalización y de ruido gaussiano antes de la función de activación y después de esta una capa de dropout del 20% y 30% para las últimas dos capas ocultas. La primera la hemos dejado sin dropout.

Los resultados tras agregar estas nuevas 2 capas ocultas con su respectiva normalización no ha sido otra cosa que decepcionante. A pesar de haber mejorado la precisión sobre el conjunto de test en un 0.08%, la precisión con respecto al conjunto de entrenamiento ha caído en más de un punto, situándose en el 91.25%

Tras múltiples pruebas variando el número de neuronas y el factor de dropout en la parte fully connected, vimos que posiblemente el problema podía no estar en esta parte. Sino que podía venir de una falta de regularización en la parte convolucional de nuestra red. Para validar nuestra hipótesis, tras cada una de las capas convolucionales agregamos un dropout progresivo a medida que profundizas en la topología de la red. Siendo menor en la entrada y mayor en la salida. Además, quitamos una de las capas ocultas de la sección fully connected, aumentando el número de neuronas de las dos capas restantes de 512 a 1024. La topología quedaba como sigue:

Conv(32) → Drop(10) → Conv(32) → MaxP(2) → Conv(64) → Drop(10) → Conv(64) → MaxP(2) → Conv(128) → Drop(20) → Conv(128) → Drop(20) → Conv(128) → MaxP(2) → Conv(256) → Drop(20) → Conv(256) → Drop(20) → Conv(256) → MaxP(2) → Conv(512) → Drop(20) → Conv(512) → Drop(20) → Conv(512) → MaxP(2) → Fcon(1024) → Drop(30) → Fcon(1024) → Drop(30) → Fcon(10) → softMax

Con este cambio los resultados ya son bastante más prometedores. El modelo presenta un 93.62% de acierto en train y un 90.72% en test.

Convencidos de que esta es la topología adecuada para llegar a nuestro objetivo, intentamos ajustar los parámetros al problema. Sabiendo que la parte encargada de extraer las características es la convolucional, decidimos aumentar el número del filtros por capa, así como el dropout de estos como medida regularizadora. Además, pasamos a reducir el tamaño de las dos capas ocultas de la parte fully connected a 512 y 256 haciendo un pequeño embudo para suavizar el paso hacia la capa de salida:

Conv(64) → Drop(20) → Conv(64) → MaxP(2) → Conv(128) → Drop(20) → Conv(128) → MaxP(2) → Conv(256) → Drop(20) → Conv(256) → Drop(20) → Conv(256) → MaxP(2) → Conv(512) → Drop(30) → Conv(512) → Drop(30) → Conv(512) → MaxP(2) → Conv(512) → Drop(30) → Conv(512) → Drop(30) → Conv(512) → MaxP(2) → Fcon(512) → Drop(40) → Fcon(256) → Drop(40) → Fcon(10) → softMax

Con los ajustes mencionados previamente obtuvimos un resultado realmente bueno. El modelo consiguió aprender a clasificar el 95.12% de las muestras de entrenamiento y el 91.57% de las de test. Como se puede ver en la figura 3, la evolución de la precisión en el conjunto de entrenamiento sí es consistente, pero para el conjunto de test es muy variable. Además, en las fases finales vemos como la precisión sobre el conjunto de test se queda estancada.

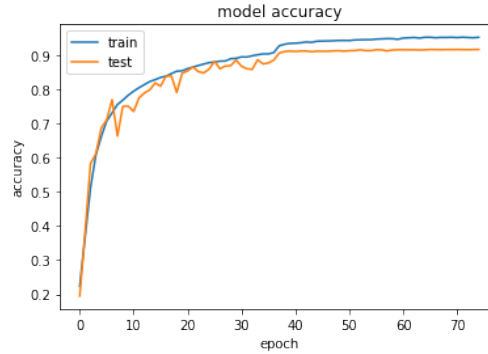


Figure 3: Evolución del entrenamiento sobre cifar

En este punto realizamos un ajuste de parámetros sobre los regularizadores del modelo. Disminuimos el valor del dropout de la primera convolución (del 20% al 10%) así como el ruido gaussiano aplicado en cada una de éstas (de 0.25 a 0.20). Además, dimos un poco más de margen al modelo para aprender, aumentando el número de épocas de 75 a 100. Con estas modificaciones llegamos al resultado esperado: 92.01%. En la figura 4 se puede observar la evolución del entrenamiento:

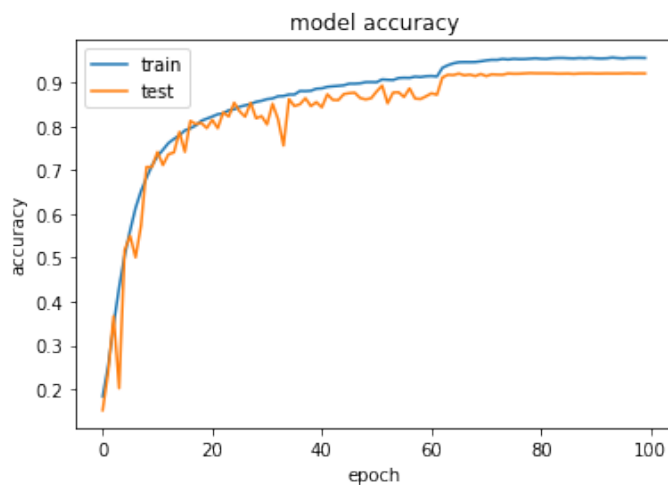


Figure 4: Evolución del mejor modelo sobre cifar

Vemos que ambas figuras tienen mucha similitud, siendo que no hemos sido capaces de resolver la oscilación de la precisión sobre el conjunto de test. No obstante el ajuste en los parámetros y el aumento de las épocas ha hecho que hayamos podido llegar al objetivo.